

Implementasi dan Analisis Komparatif WebSocket dan HTTP Short Polling pada Backend Go Fiber terhadap Latensi, Penggunaan CPU, dan Memori

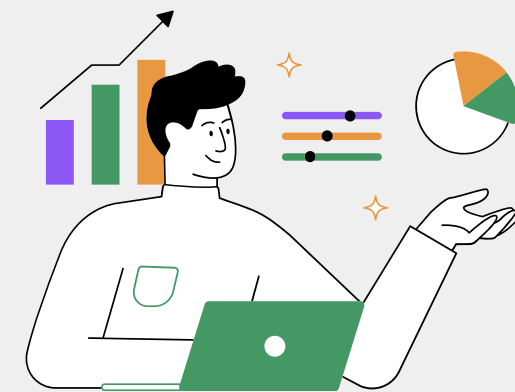
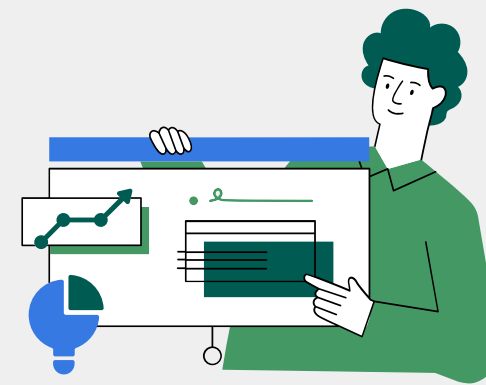
MUHAMMAD FARIS ASSAMI



Dosbim : AJIB SUSANTO, M.Kom



Latar Belakang



Aplikasi web modern menuntut penyajian data secara instan tanpa reload halaman. Platform kolaborasi membutuhkan fitur live chat dan push notification.

Teknik polling tradisional memiliki overhead tinggi (500–800 bytes/header), pemborosan CPU & RAM akibat request repetitif, dan latensi signifikan karena setiap siklus memerlukan koneksi HTTP baru.

WebSocket menawarkan koneksi full-duplex persisten dengan overhead minimal (2–14 bytes). Go Fiber memanfaatkan goroutines (~2KB stack) untuk menangani konkurensi tinggi secara efisien.

Penelitian sebelumnya menggunakan PHP/JavaScript dengan keterbatasan konkurensi. Belum ada eksplorasi perbandingan kinerja kuantitatif WebSocket vs HTTP Short Polling pada backend Go Fiber.

Kebutuhan Real-time

Masalah HTTP Short Polling

Alternatif: WebSocket + Go Fiber

Gap Penelitian

Rumusan Masalah & Batasan Masalah ↗



Rumusan Masalah

- * Bagaimana mengimplementasikan fitur real-time chat dan notifikasi pada platform Synergazing menggunakan WebSocket dengan Go Fiber?
 - * Bagaimana kinerja WebSocket ditinjau dari latensi, CPU, dan memori?
 - * Bagaimana perbandingan kinerja WebSocket dan HTTP Short Polling?
-



Batasan Masalah

- ” Backend: Go (Golang) + Framework Fiber, Frontend: Next.js
- ” Fitur real-time terbatas pada chat dan notifikasi pada platform Synergazing
- ” Metrik kinerja: latensi, CPU, dan memori — diuji dalam lingkungan terkontrol (Docker)
- ” Tidak membahas: keamanan (E2E encryption), desain UI/UX, maupun deployment skala produksi

Tujuan Penelitian

1

Mengimplementasikan layanan backend real-time (chat & notifikasi) dengan WebSocket + Go Fiber

2

Menganalisis performa sistem: latensi, CPU, dan memori

3

Membuktikan secara eksperimental keunggulan efisiensi WebSocket dibanding HTTP Short Polling

Teoritis

- Menjadi referensi akademis implementasi WebSocket pada Go Fiber & menambah literatur perbandingan push-based vs pull-based

Praktis

- Bagi Penulis: Meningkatkan kompetensi backend modern (Go, WebSocket)
- Bagi Platform: Menghasilkan prototipe chat & notifikasi yang fungsional dan efisien
- Bagi Pengembang: Menyediakan studi kasus dan panduan praktis implementasi serupa

Manfaat penelitian ↗

Tinjauan Studi

Novelty

- Implementasi WebSocket pada Go Fiber (goroutine-based, konkurensi tinggi)
- Analisis 3 metrik sekaligus: latensi, CPU, dan memori
- Komparasi WebSocket vs HTTP Short Polling pada kondisi identik
- Studi kasus platform kolaborasi riil dengan 2 fitur (chat + notifikasi)

peneliti	judul	teknologi	hasil utama	gap
Alviando et al. (2023)	Penerapan WebSocket pada Sistem Live Chat (Studi Kasus Kwikku.com)	PHP/JavaScript	WS 52% lebih efisien; pada 200 user: WS 233 ms vs LP 857 ms	Belum eksplorasi Go untuk konkurensi tinggi
Sathya & Swetha (2025)	Real-Time Chat Application with Web Socket for Network Efficiency	Umum	WS 35 ms vs AJAX 220 ms, bandwidth -25%, CPU turun 40%	Tanpa pengukuran mendalam pada framework Go
Suwarjono & Adna (2025)	Pengembangan Aplikasi Antrian Berbasis WebSocket Menggunakan PHP	PHP Ratchet	WS 84 ms vs Polling 1987 ms; stabil hingga 100 koneksi	PHP single-threaded, belum goroutines
Diyasa et al. (2022)	Push Notification Using the Websocket (Studi Kasus SITUK-V2)	Fungsional	Push notification delay 0.18–0.59 detik	Tanpa analisis beban CPU/RAM server
Ardiansyah & Fatwanto (2022)	Comparison of Memory Usage between REST API in Javascript and Golang	Go vs JS	Selisih memori hanya 0.07–2.94 MB (< 0.4% kapasitas)	Hanya REST API, belum WebSocket
Malhotra (2025)	Concurrency Patterns in Golang: Real-World Use Cases	Pola Konkurensi Go	Throughput 3.5x lipat, worker pools memori 83% lebih stabil	Benchmark teoritis, belum implementasi WebSocket riil

Kebutuhan Real-time

- ✿ Transmisi data instan dengan latensi minimal, tanpa reload halaman
- ✿ Esensial untuk live chat, push notification, game multiplayer, dan IoT



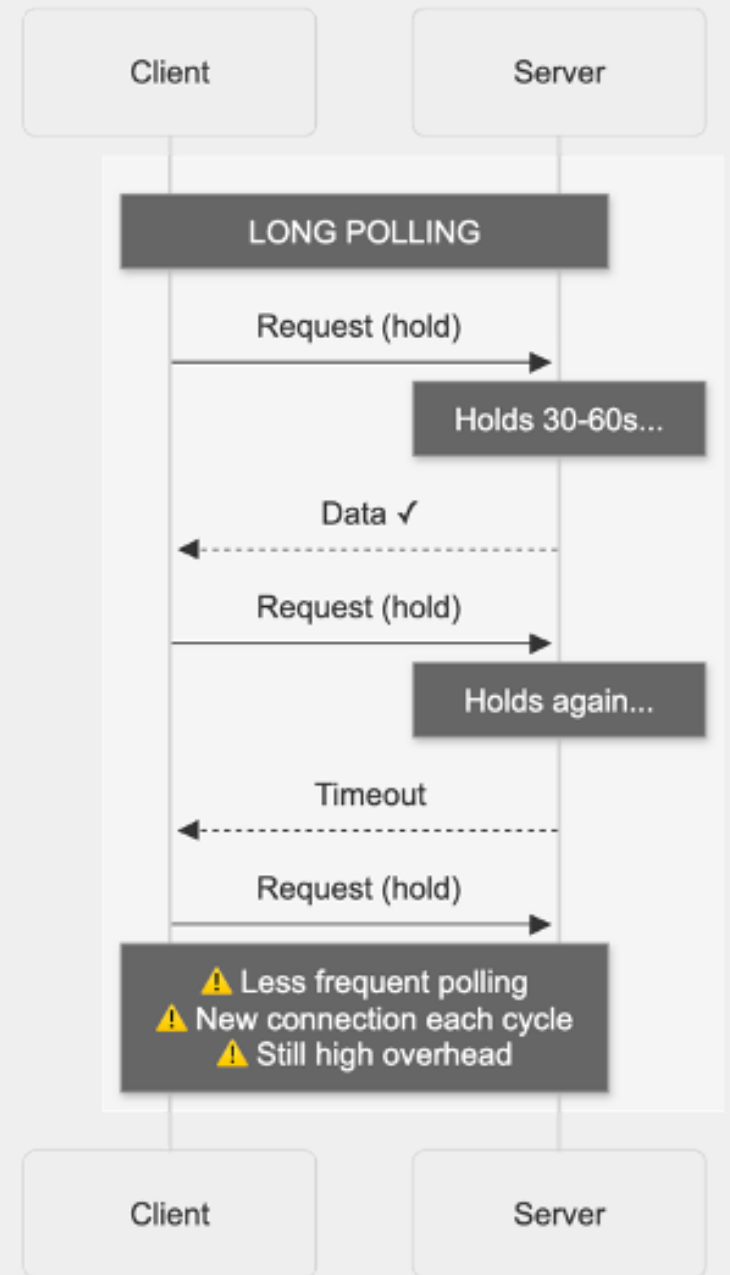
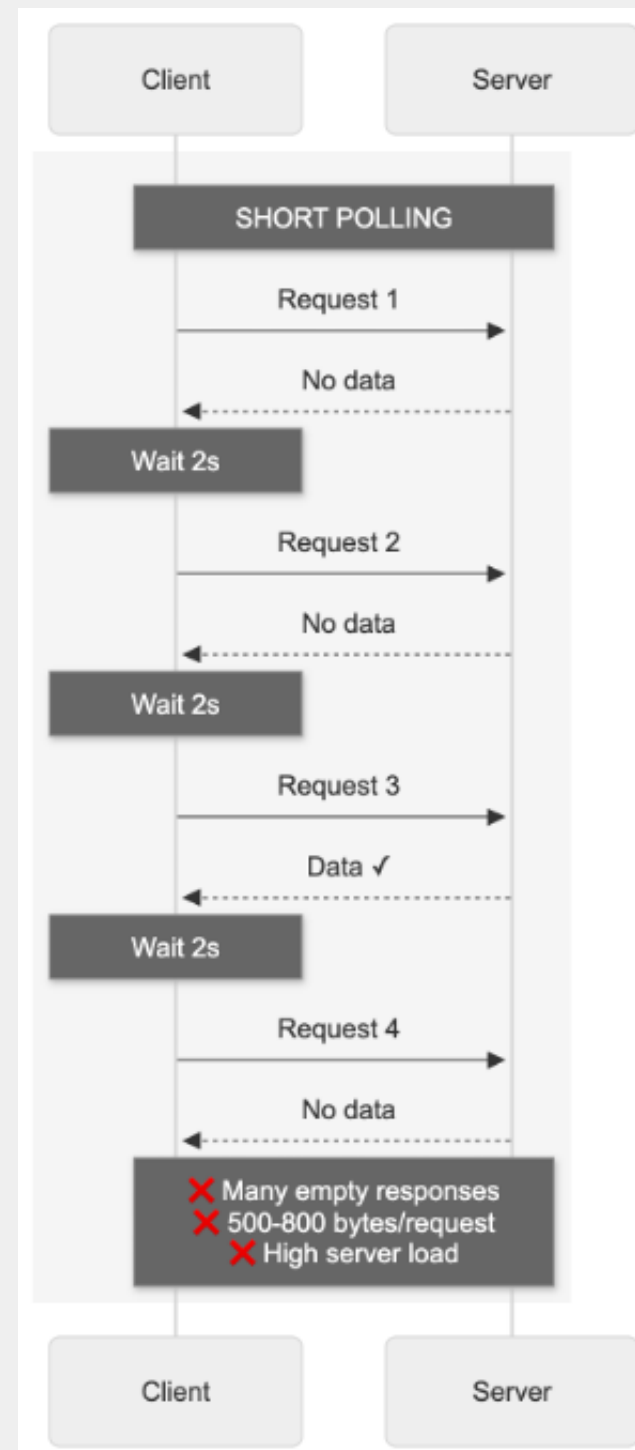
Metode Polling

Short Polling

- Klien mengirim request periodik (misal tiap 2 detik)
- banyak request kosong, overhead header 500–800 bytes

Long Polling

- Server menahan koneksi hingga data tersedia
- tetap overhead tinggi karena harus membuat koneksi HTTP baru tiap siklus



Go & Framework Fiber

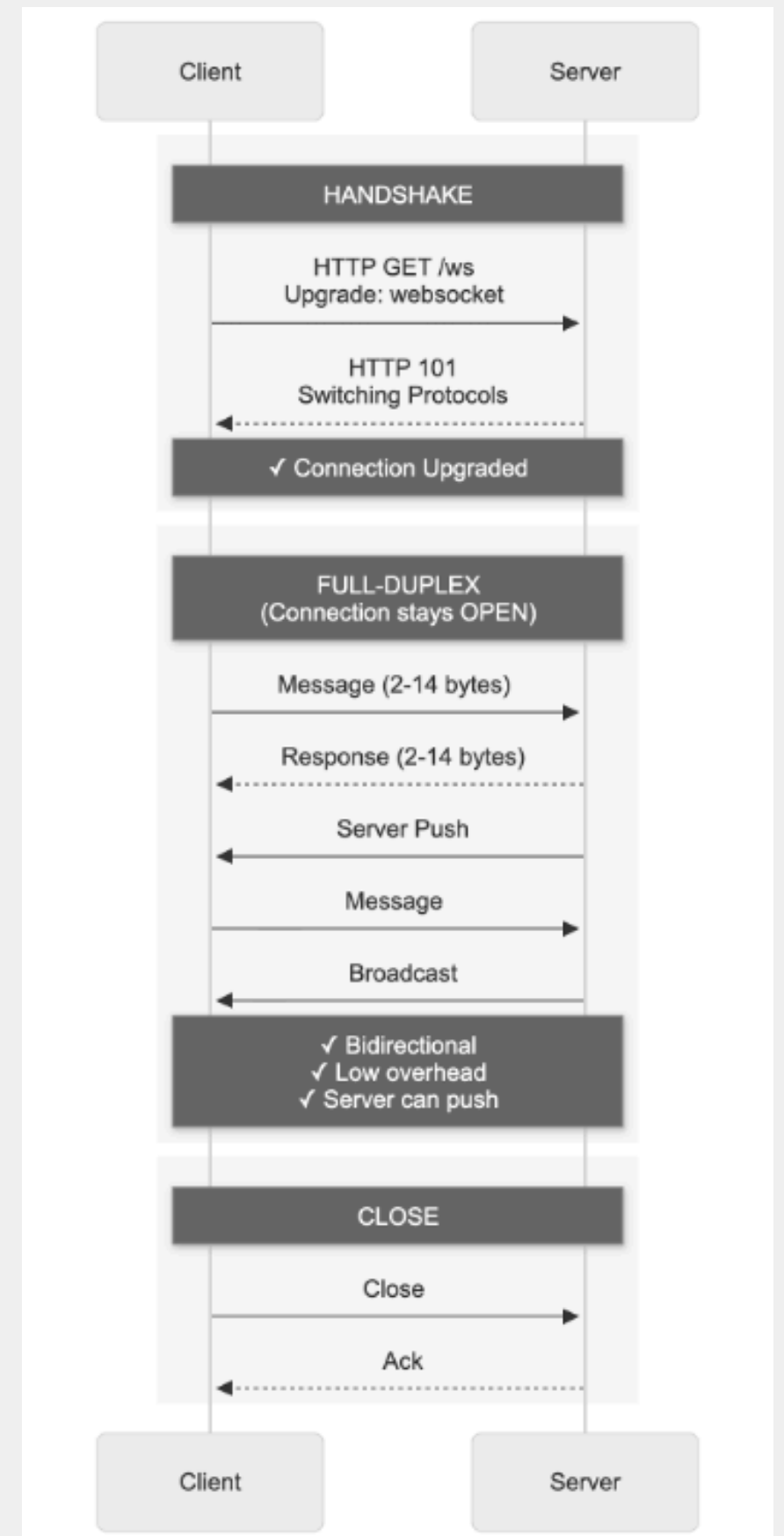
- Goroutines: ~2KB stack per goroutine vs 1–2MB thread OS → mampu menangani jutaan koneksi simultan
- Fiber dibangun di atas Fasthttp dengan zero memory allocation routing
- Benchmark TechEmpower: Fiber 2,4 juta req/s vs Express.js 301K req/s

Analisis Kinerja

- ➔ Latensi adalah Waktu pesan dari pengirim ke penerima dalam milidetik
- ➔ CPU & Memori: Beban sumber daya server saat menangani koneksi

Protokol WebSocket

- * Komunikasi full-duplex melalui 1 koneksi TCP persisten
- * Dimulai dengan HTTP handshake → server respons 101 Switching Protocols → koneksi tetap terbuka
- * Server dapat push data kapan saja tanpa diminta (server-sent push)
- * Overhead frame minimal: 2–14 bytes vs 500–800 bytes HTTP header



Metodologi Penelitian & Pengembangan

01 Requirements Planning

Spesifikasi kebutuhan & metrik kinerja

1-2W

02 User Design

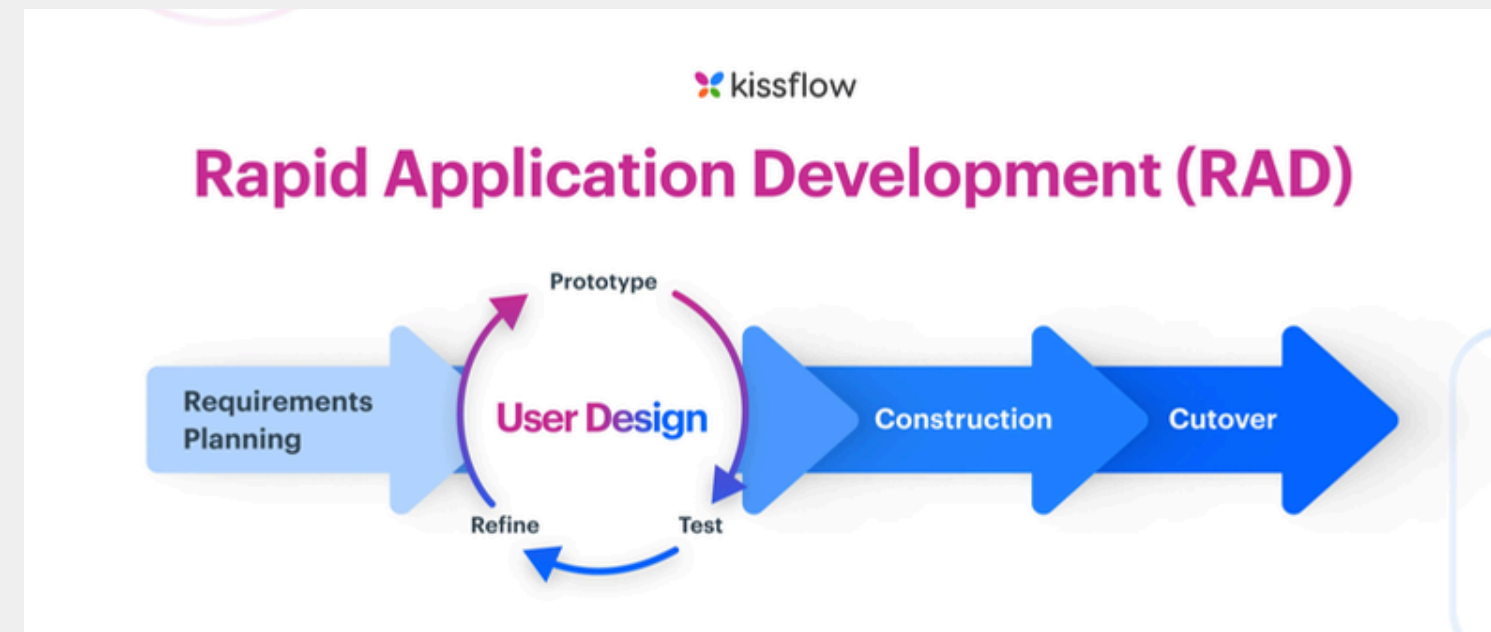
ERD, arsitektur WebSocket, API spec

3-4W

03 Construction 1

Backend Go Fiber + WebSocket handler

5-7W



- ✦ Metode Penelitian: Eksperimental dengan pendekatan kuantitatif
- ✦ Metode Pengembangan: Rapid Application Development (RAD)

05 Cutover

Load testing & analisis data

11-14W

04 Construction 2

Frontend Next.js + HTTP Short Polling

8-10W

Arsitektur Sistem

5 Layer Arsitektur



Client Layer

Frontend Next.js (React + TypeScript)



Communication Layer

WebSocket (persistent & full-duplex) + HTTP REST API



Backend Layer

Go Fiber — Chat/Notification Controllers dengan Goroutines



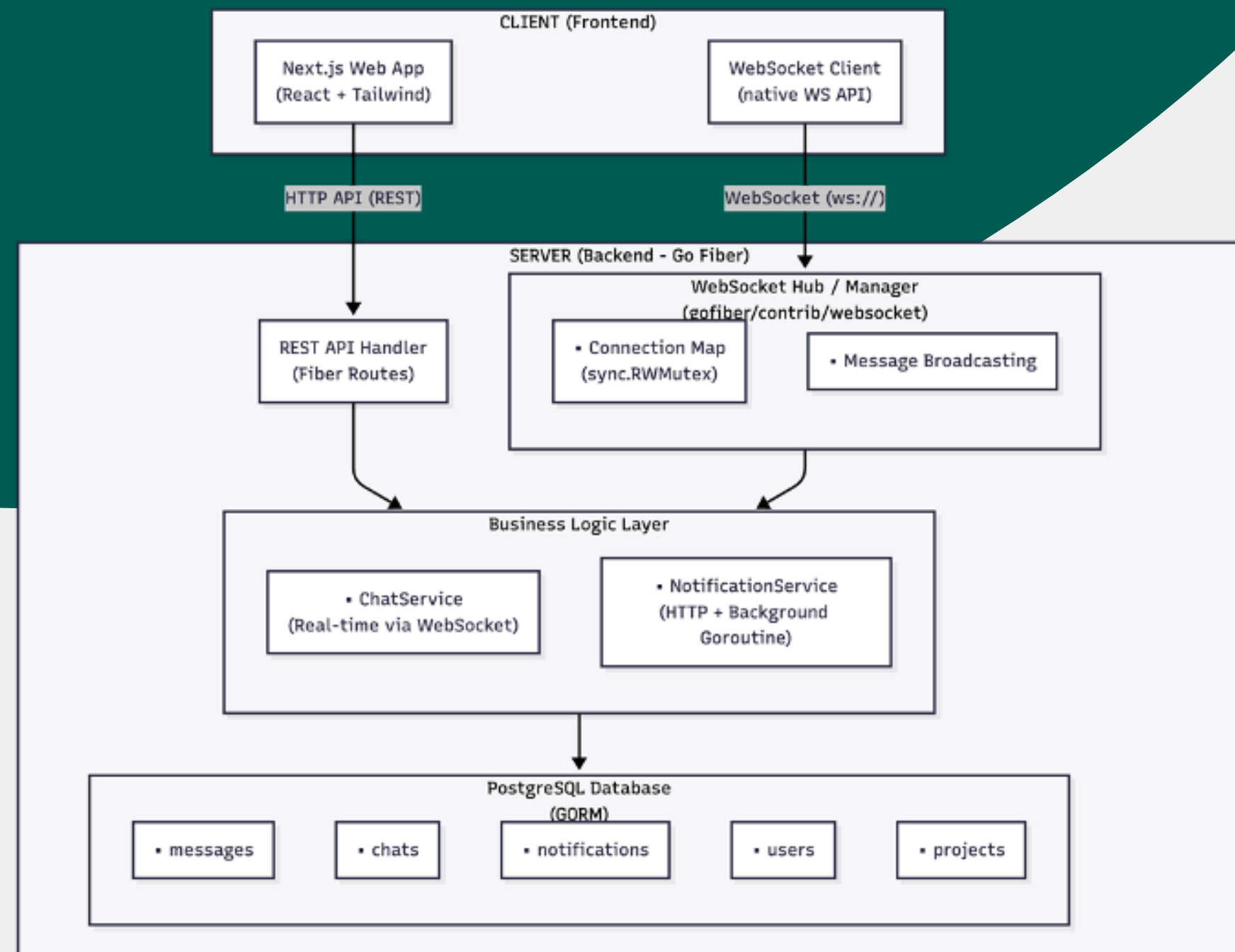
Data Layer

PostgreSQL untuk persistensi Users, Chats, Messages, Notifications



Monitoring Layer

Prometheus & Grafana untuk visualisasi metrik real-time

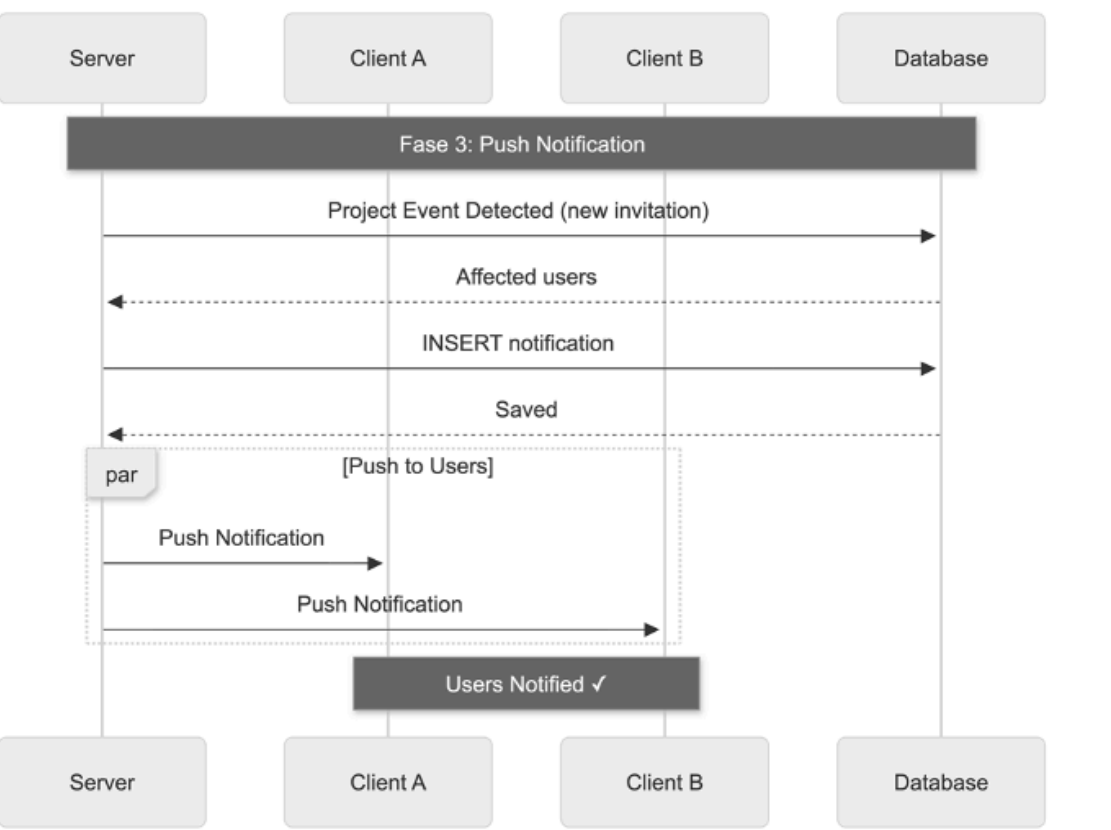
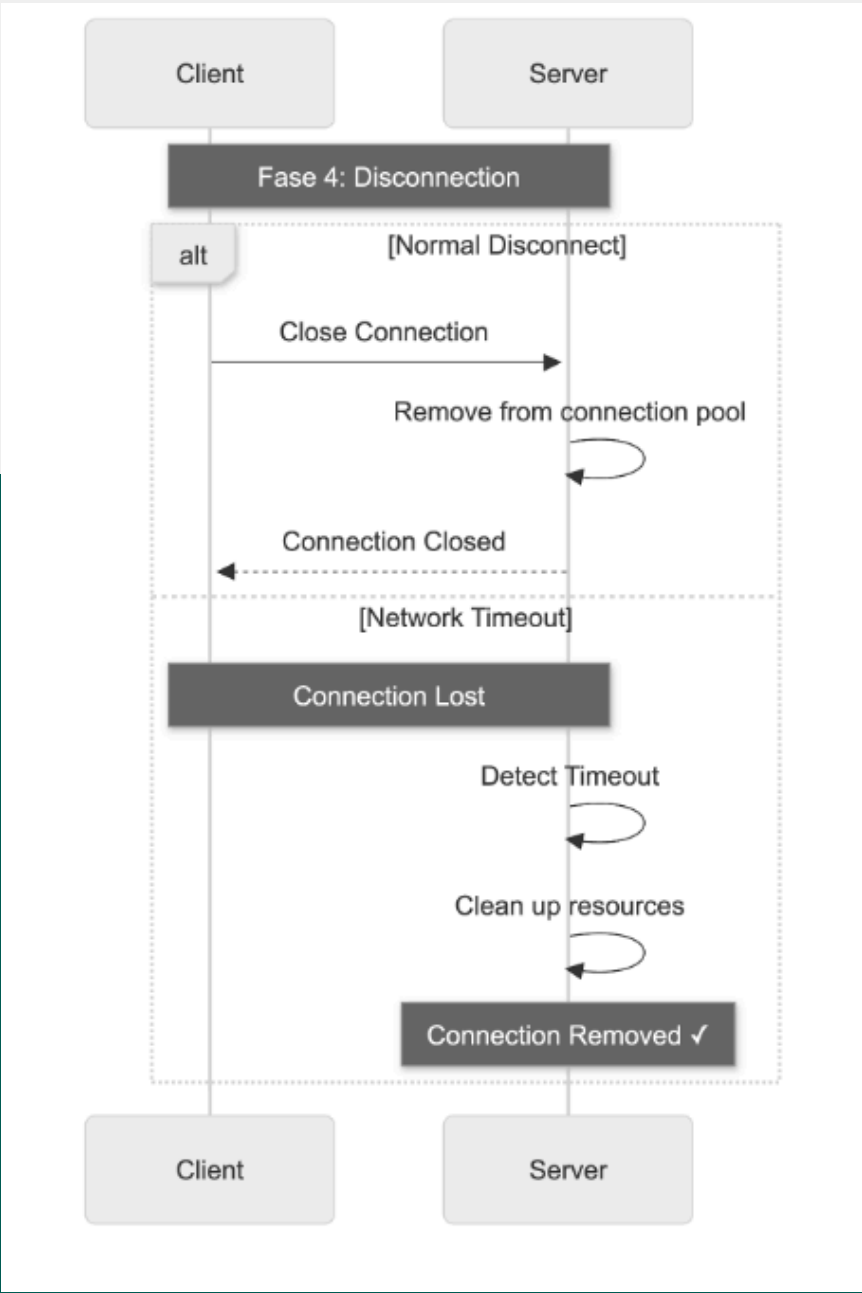
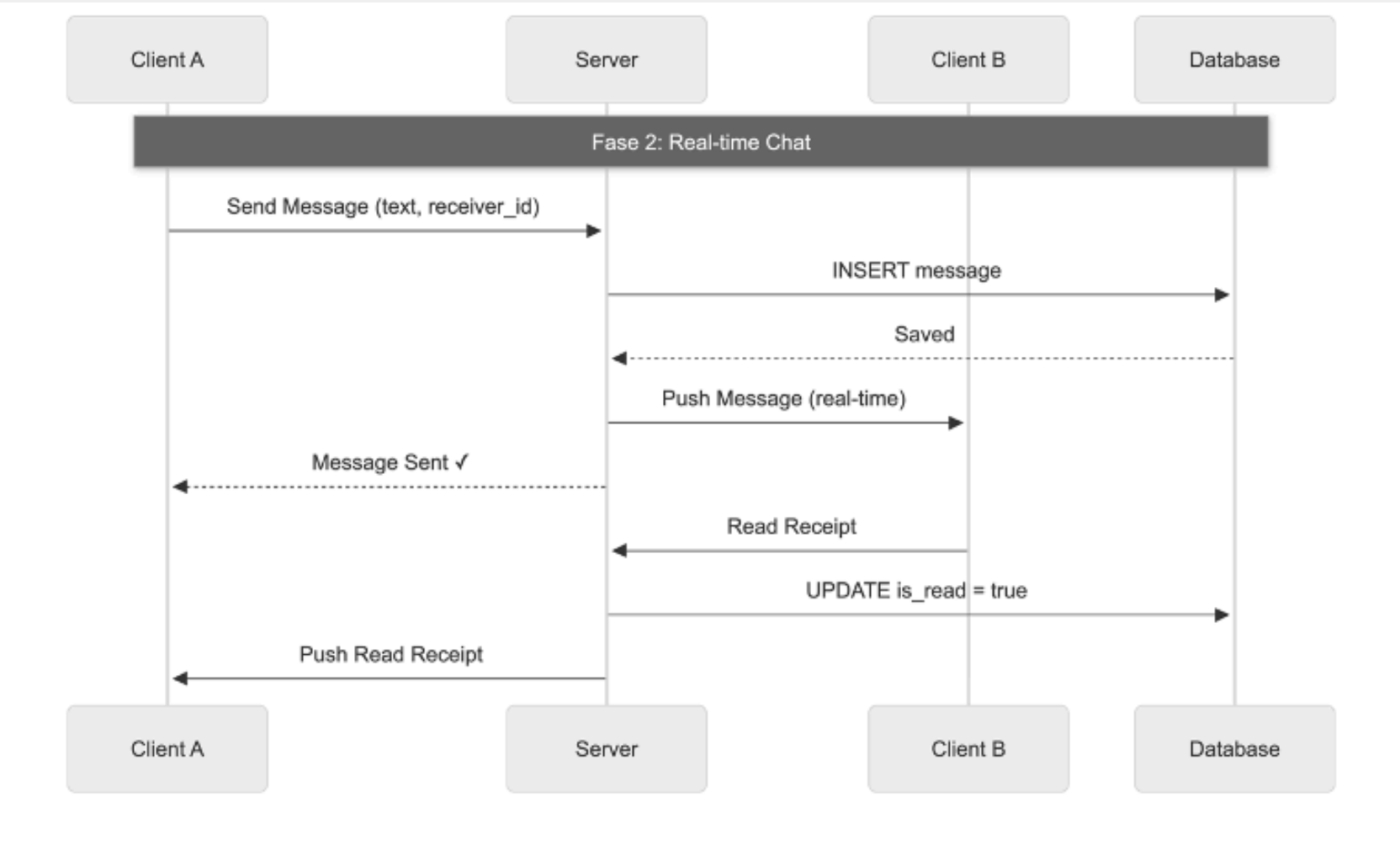
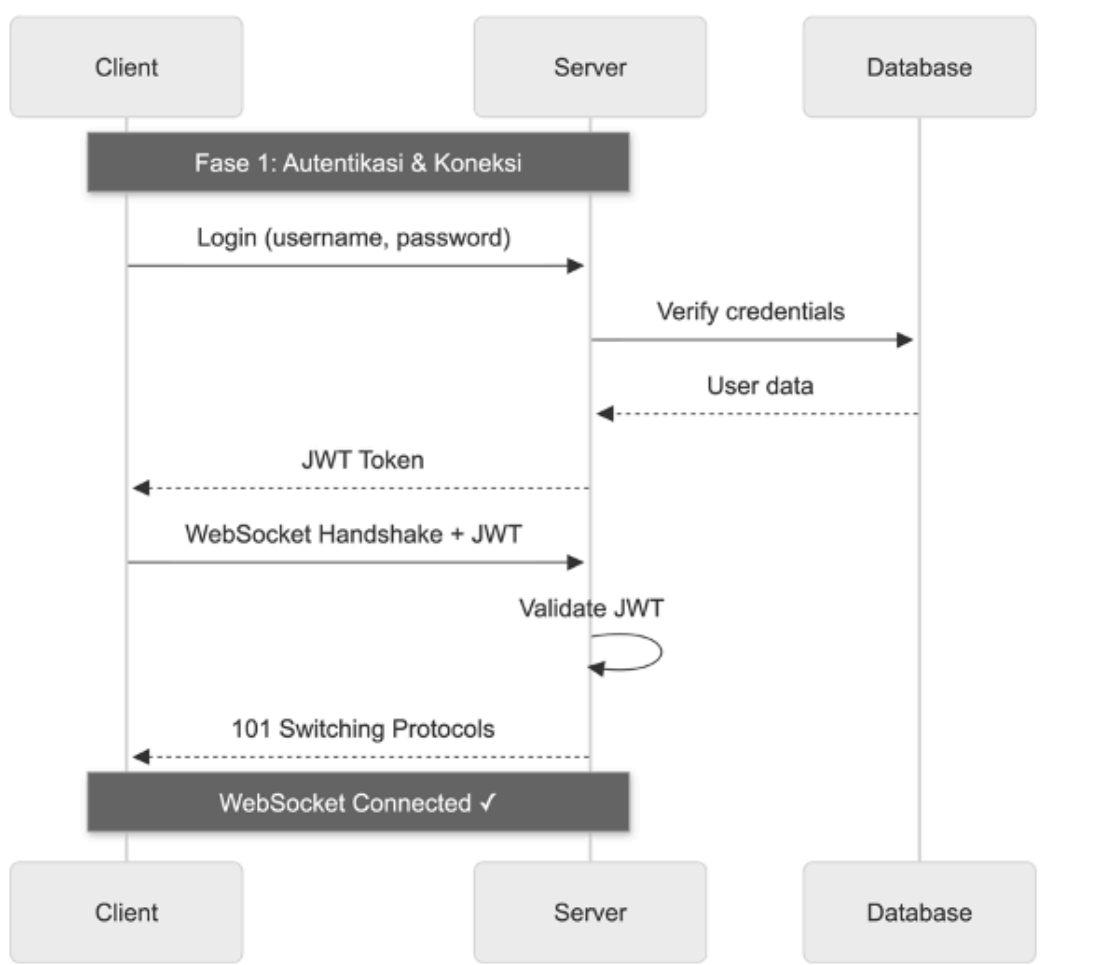


Entitas Utama

Entitas	Deskripsi	Atribut Utama
users	Data pengguna sistem	id, username, email, password_hash, full_name
projects	Proyek kolaborasi	id, project_name, description, owner_id
chats	Ruang percakapan antar pengguna	id, user_one_id, user_two_id, last_message_at
messages	Pesan chat	id, chat_id, sender_id, message_text, is_read
notifications	Notifikasi pengguna	id, user_id, notification_type, title, is_read



Sequence Diagram



IMPLEMENTASI & PENGUJIAN FUNGSIONAL

Implementasi Backend

-  WebSocket Handler: thread-safe connection management (sync.RWMutex)
-  Push Notification: broadcasting ke multiple users via WebSocket
-  Autentikasi JWT untuk keamanan koneksi
-  PostgreSQL + GORM untuk persistensi data

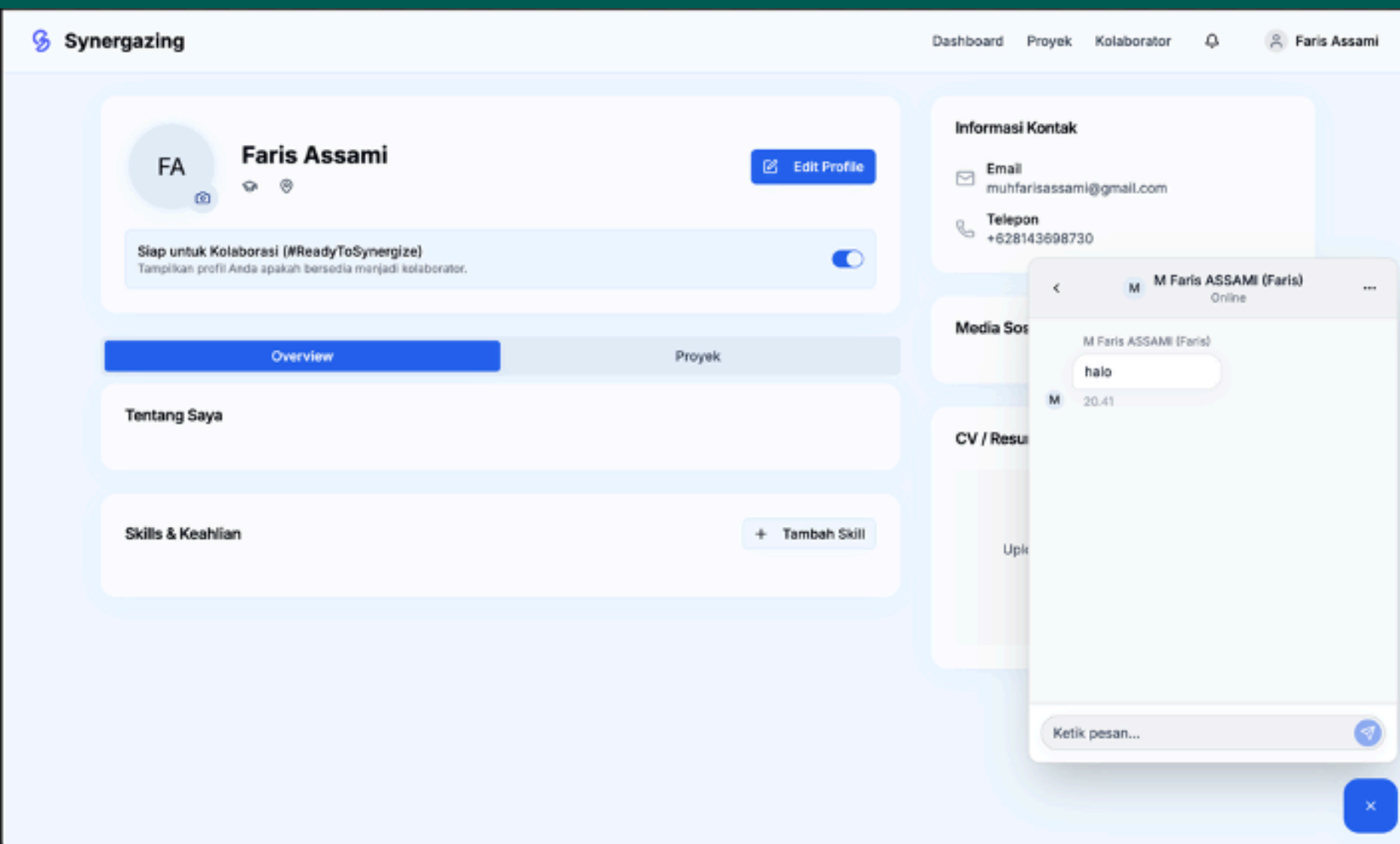
Implementasi FrontEnd

-  Native WebSocket API dengan automatic reconnection
-  UI Chat real-time & Panel Notifikasi

Blackbox Testing: 15/15 Test Cases PASSED ✓ (100%)

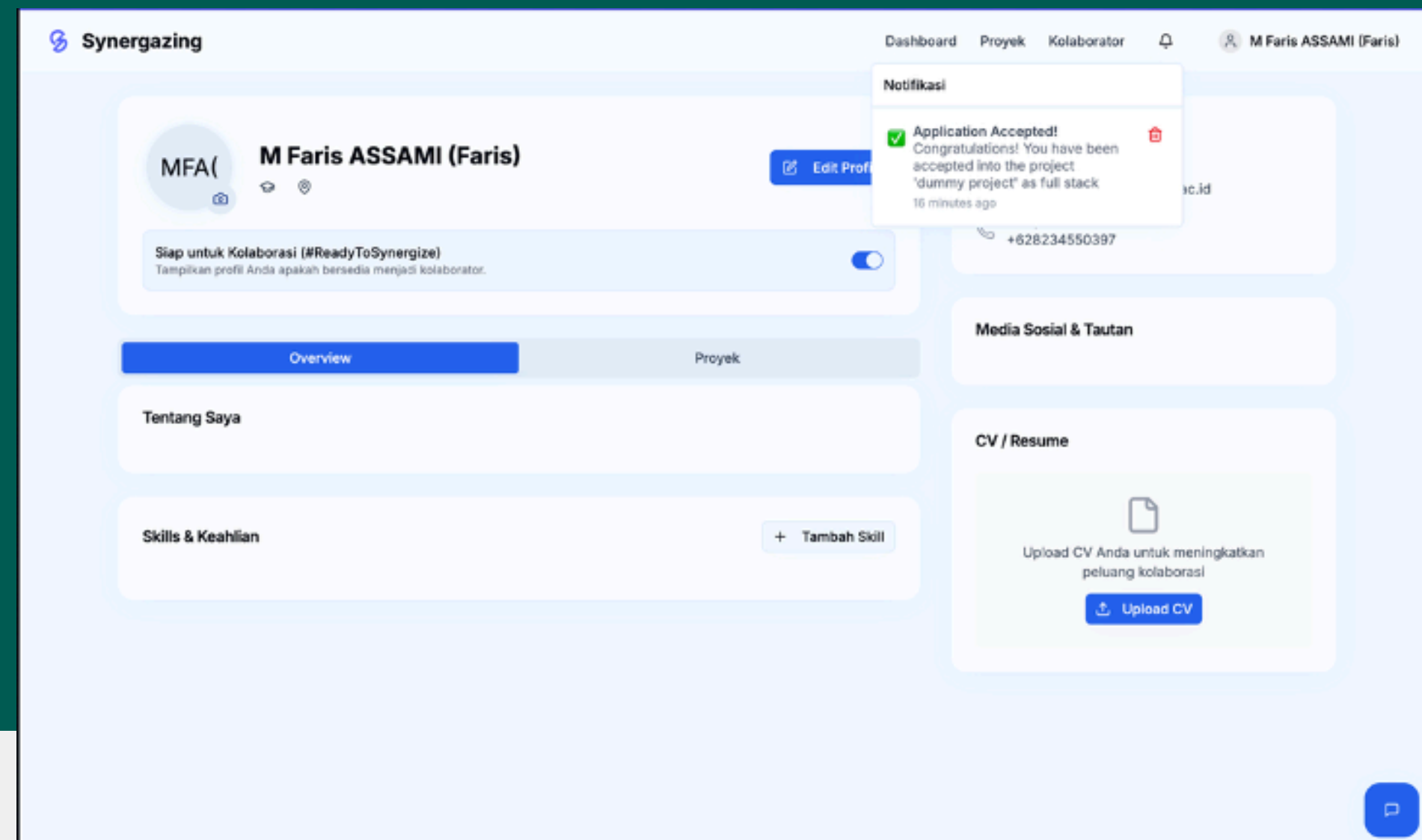
Entitas	Deskripsi	Atribut Utama
Autentikasi (TC-01 ~ TC-03)	Login valid/invalid/empty	✓ Pass
WebSocket Connection (TC-04 ~ TC-06)	Token valid/no token/expired	✓ Pass
Real-time Chat (TC-07 ~ TC-09)	Send online/offline, read receipt	✓ Pass
Push Notification (TC-10 ~ TC-12)	Trigger, broadcast, mark read	✓ Pass
Disconnection (TC-13 ~ TC-15)	Normal, timeout, reconnection	✓ Pass

Hasil Implementasi UI



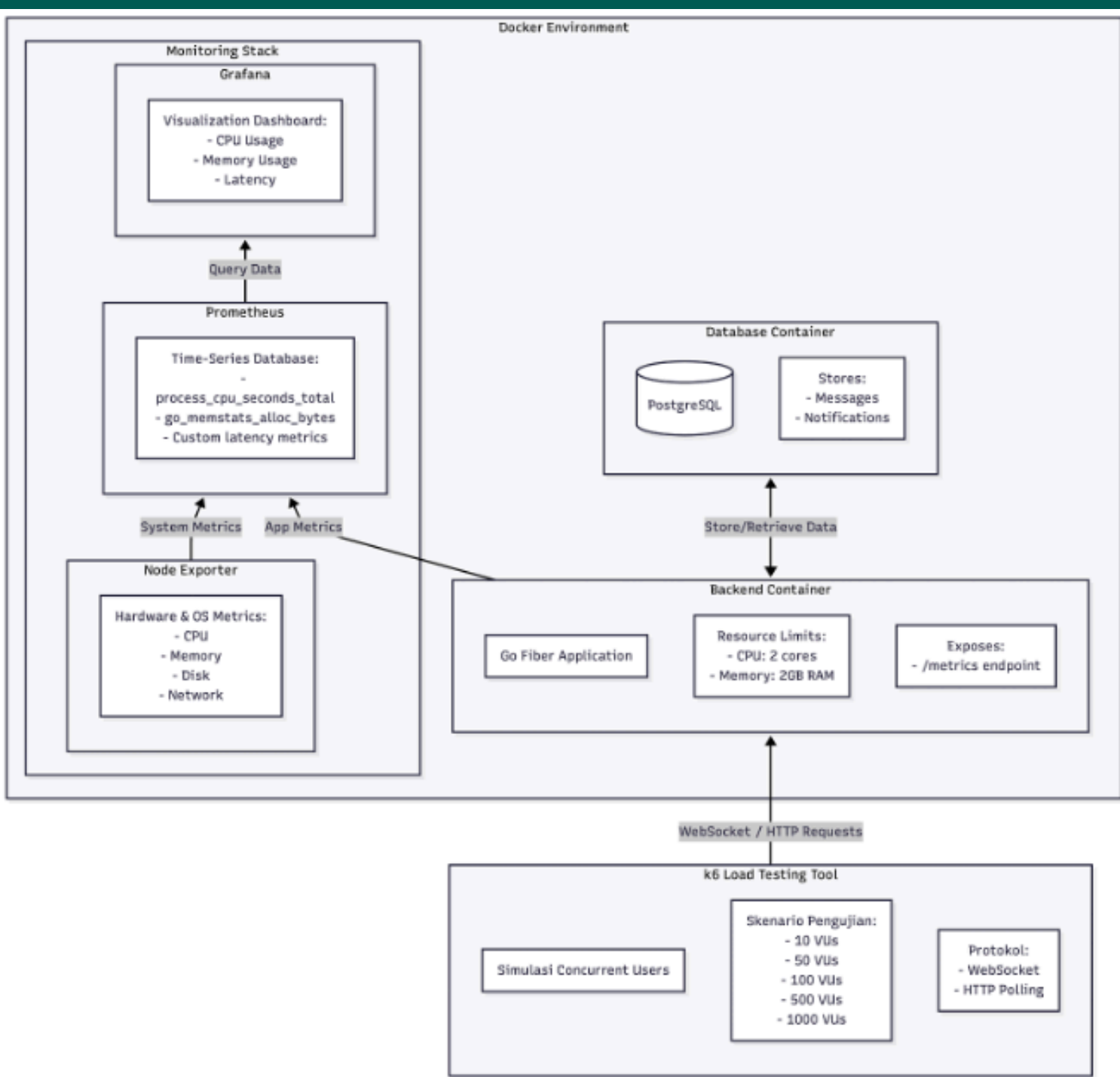
* Fitur Real-time Chat

- ✦ Panel chat dengan status koneksi WebSocket ("Terhubung")
- * Bubble chat menampilkan pesan yang dikirim dan diterima secara real-time
- ▣ Read receipt dan timestamp otomatis



* Fitur Notifikasi

- ▣ Notifikasi diterima secara instan melalui WebSocket tanpa polling
- ▣ Contoh: notifikasi "Application Accepted!" untuk undangan proyek
- ▣ Badge counter yang update secara real-time



Infrastruktur & Skenario Load Testing ↗



Infrastruktur Pengujian

System Under Test: Backend Go Fiber dalam Docker Container (2 CPU, 4GB RAM)

Load Generator: k6 (di luar container — menghindari resource contention)

Monitoring: Node Exporter → Prometheus → Grafana



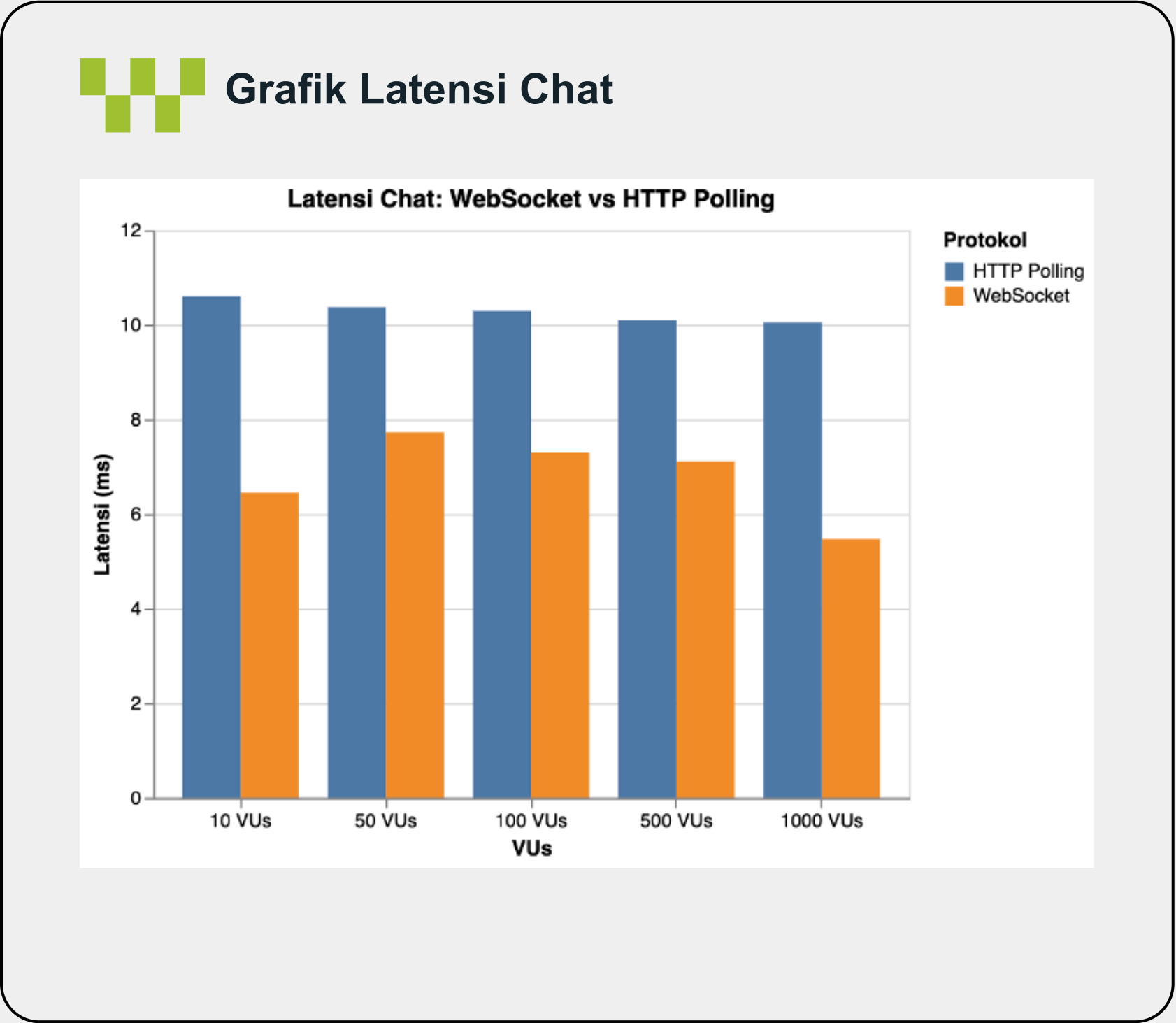
Skenario Pengujian (Websocket & HTTP Polling)

Variasi beban: 10, 50, 100, 500, 1000 Virtual Users (VUs)

100 pesan per koneksi, durasi 5–10 menit per skenario

Metrik: Latensi (ms), CPU (%), Memori (MB)

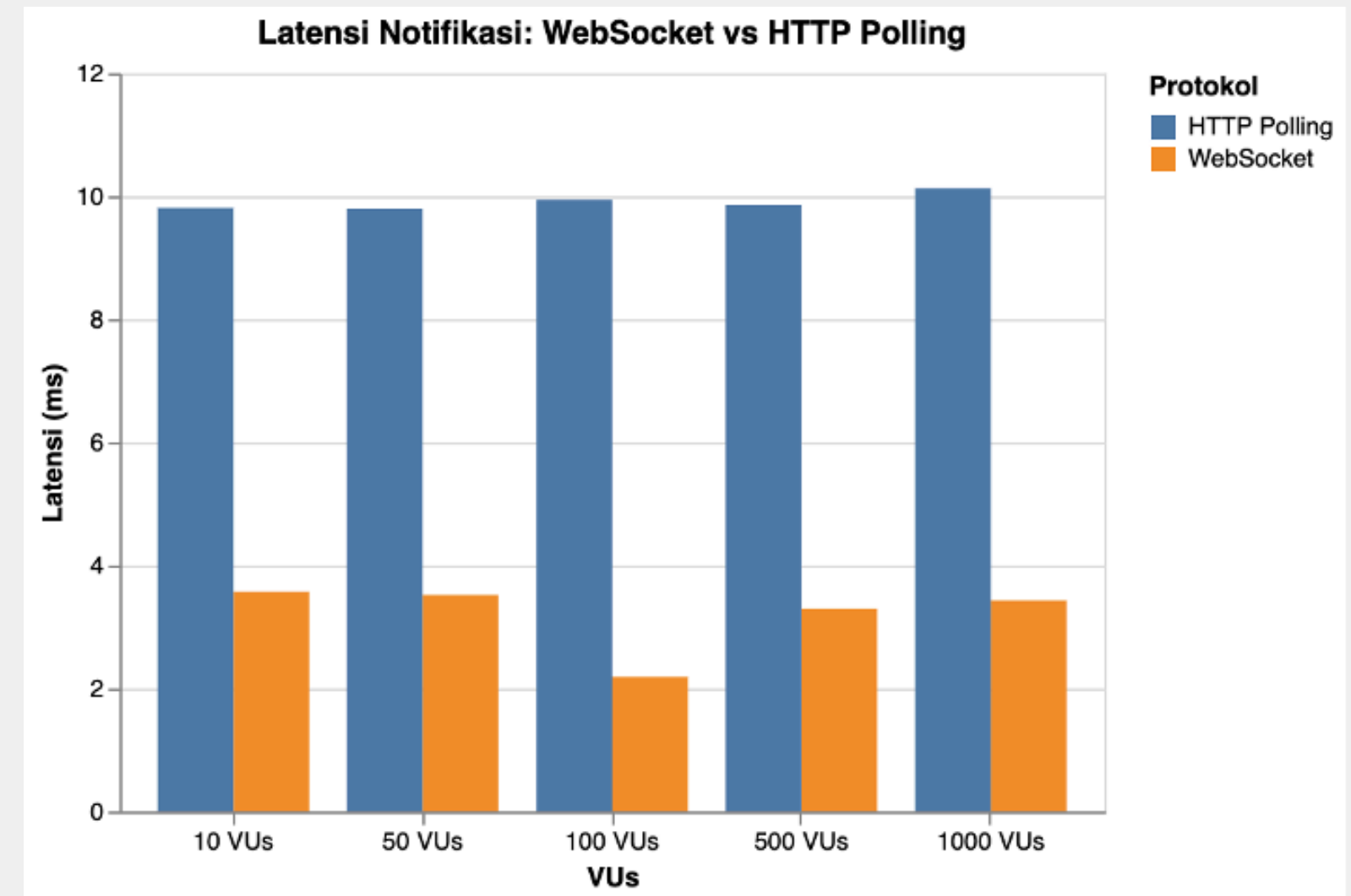
Jumlah VUs	WS Avg (ms)	WS p95 (ms)	Polling Avg (ms)	Polling p95 (ms)	Peningkatan
10 VUs	6.45	9.55	10.60	13.0	39.2% Lebih Cepat
50 VUs	7.72	11.0	10.37	13.15	25.6% Lebih Cepat
100 VUs	7.30	9.0	10.29	13.2	29.1% Lebih Cepat
500 VUs	7.11	12.1	10.10	13.0	29.6% Lebih Cepat
1000 VUs	5.47	9.0	10.05	13.15	45.6% Lebih Cepat



Jumlah VUs	WS Avg (ms)	WS p95 (ms)	Polling Avg (ms)	Polling p95 (ms)	Peningkatan
10 VUs	3.57	5.0	9.81	12.0	63.6% Lebih Cepat
50 VUs	3.52	5.0	9.80	12.05	64.1% Lebih Cepat
100 VUs	2.19	4.0	9.94	13.0	78.0% Lebih Cepat
500 VUs	3.30	4.0	9.86	13.0	66.5% Lebih Cepat
1000 VUs	3.43	4.3	10.13	13.0	66.1% Lebih Cepat

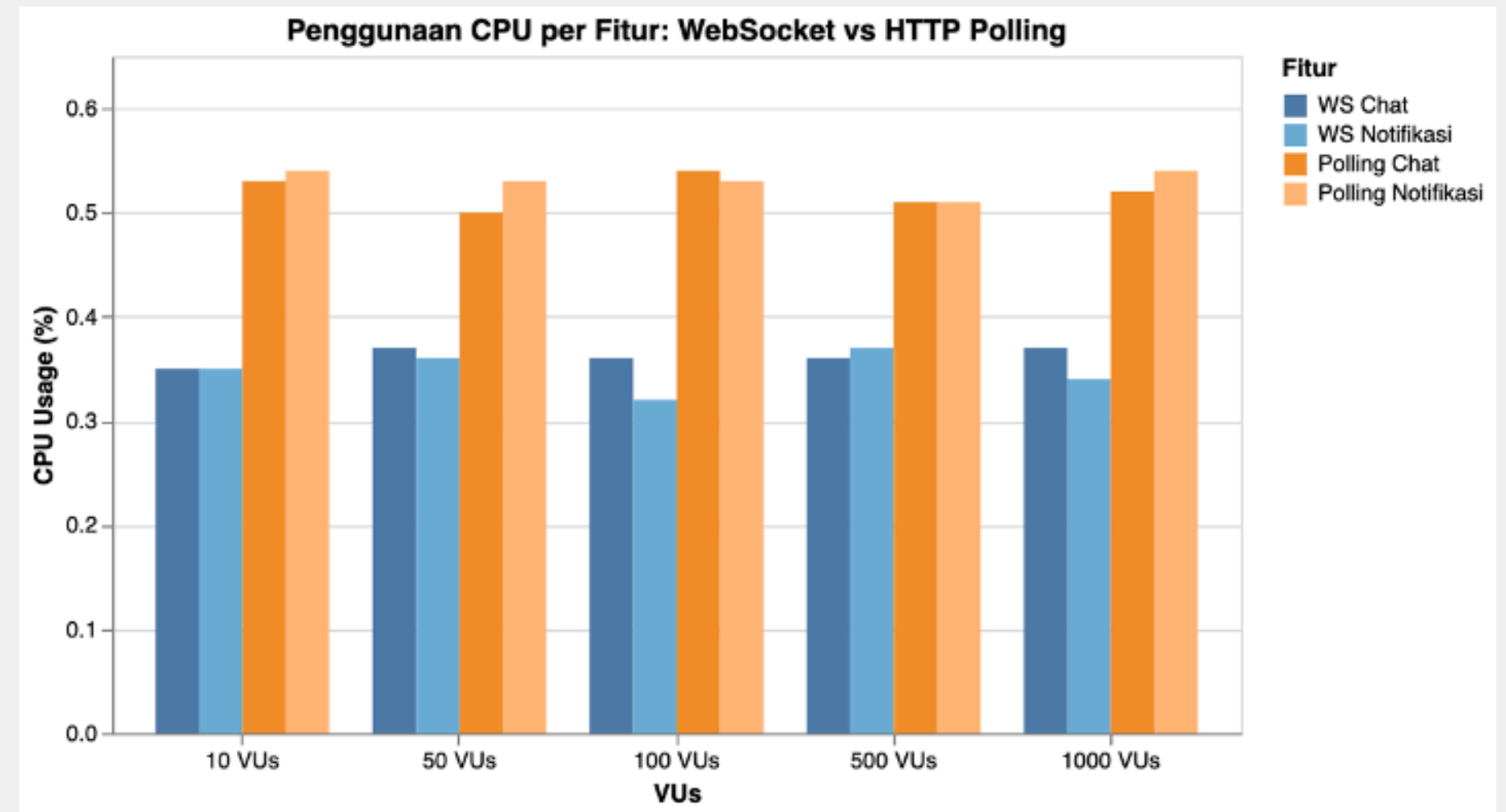


Grafik Latensi Notifikasi



Universitas Dian Nuswantoro

Grafik Penggunaan CPU



Ringkasan Perbandingan Kinerja

Metrik	Websocket	HTTP Short Polling	Kesimpulan
Latensi Chat	5.47–7.72 ms	10.05–10.60 ms	WS 26–46% lebih cepat
Latensi Notifikasi	2.19–3.57 ms	9.80–10.13 ms	WS 63–78% lebih cepat
CPU Usage	0.32–0.37%	0.50–0.54%	WS 32% lebih efisien
Memory Usage	~7 MB	~6 MB	Serupa (~1 MB selisih)
Koneksi	Persisten (1 koneksi)	Request berulang/detik	WS lebih efisien

Skalabilitas WebSocket

Tingkat Beban	Latensi Chat	Perubahan dari Baseline	Status
10 VUs (Baseline)	6.45	-	Baik
50 VUs	7.72	+19.7%	Baik
100 VUs	7.30	+13.2%	Baik
500 VUs	7.11	+10.2%	Optimal
1000 VUs	5.47	-15.2% (membaik)	Optimal

Kesimpulan

RQ1 Implementasi

- Sistem real-time chat & notifikasi berhasil dibangun dengan Go Fiber + WebSockets
- 15/15 blackbox test cases passed (100%)

Kesimpulan utama:

Kombinasi WebSocket dan Go Fiber merupakan solusi optimal untuk membangun fitur komunikasi real-time yang efisien dan skalabel.

RQ2 Kinerja

- Latensi chat: 5.47–7.72 ms | Latensi notifikasi: 2.19–3.57 ms
- CPU: ~0.35% | Memory: ~7 MB → sangat efisien

RQ3 Hasil WebSocket

- Latensi 26–78% lebih rendah
- CPU 32% lebih efisien
- Skalabilitas excellent (latensi membaik pada beban tinggi)

Thank You

